

1. Contexto Organizacional y Diagnóstico de la Crisis Operativa

- **Evolución de la Empresa:** Explicación del crecimiento de la academia (de cohortes iniciales de 50 estudiantes a miles de alumnos e instructores a nivel internacional).
- **Análisis de los 3 Puntos Críticos de la Crisis:**
 1. **Falta de Integridad de Datos:** Registros duplicados de cédulas/IDs, errores tipográficos en nombres y asignación incorrecta de programas debido al manejo manual en Excel (Alumnos_y_Profesores_General.xlsx).
 2. **Falta de Persistencia Confiable:** Inexistencia de bases de datos o almacenamiento en archivos de texto estructurados; riesgo permanente de pérdida de información por corrupción del archivo Excel centralizado o cierres inesperados.
 3. **Cuello de Botella en Certificación:** Proceso manual e ineficiente para determinar graduandos, calcular promedios según la modalidad académica y copiar nombres al Departamento de Certificados, lo que genera retrasos de semanas.

2. Requerimientos del Producto Mínimo Viable (MVP - SGA-DO Core)

- **Requerimientos Funcionales (RF-01 a RF-07):**
 - **RF-01 & RF-02:** Registro inmediato de Alumnos y Profesores con persistencia en alumnos.txt y profesores.txt.
 - **RF-03 & RF-04:** Registro de calificaciones y mecanismo de desapilado **LIFO (Pila / Stack)** tipo *Ctrl+Z* para deshacer la última nota en caso de error.
 - **RF-05:** Evaluación masiva de aprobación con encolado **FIFO (Cola / Queue)** y exportación a certificados_pendientes.txt.
 - **RF-06 & RF-07:** Reporte general consolidado en pantalla y salida segura con limpieza de memoria.

- **Requerimientos No Funcionales (RNF):**
 - Multi-lenguaje (Python, Java, C++) con la misma arquitectura.
 - Persistencia inmediata en archivos planos .txt.
 - Resiliencia e inviolabilidad mediante captura de excepciones (try-catch / *Negative Testing*).
 - Arquitectura Orientada a Objetos limpia con Polimorfismo estricto.

3. Arquitectura de Software y Reglas de Negocio Polimórficas

- **Jerarquía de Entidades:** Clase base Persona → Clases derivadas Alumno y Profesor.
- **Polimorfismo en Programas Académicos:** Clase base abstracta ProgramaAcademico con implementación del método evaluarAprobacion() en:
 - **Curso:** Promedio $\geq 10.0 / 20.0$.
 - **Diplomado:** Promedio $\geq 14.0 / 20.0$.
 - **Bootcamp:** Exigencia estricta donde **ninguna nota individual puede ser menor a 14.0** (si tiene una sola nota de 13.0, queda reprobado independientemente de si su promedio es alto).
- **Estructuras de Datos Lineales:** Uso explícito de **Pila (Stack / LIFO)** para deshacer notas y **Cola (Queue / FIFO)** para procesar certificados en orden de aprobación.

4. Plan Lógico de Trabajo y Cronograma

- **Fases del Proyecto (Módulos 1 al 7):**
 1. Análisis del Problema y Plan de Acción (Documento Actual)
 2. Modelado Orientado a Objetos (UML)
 3. Repositorio Git y GitHub (/docs, /python, /java, /cpp)
 4. Implementación en Python

5. Implementación en Java
6. Implementación en C++ (Gestión dinámica de memoria con new/delete y fstream)
7. Informe Ejecutivo y Presentación Final

5. Plan de Pruebas y Aseguramiento de Calidad (Protocolo QA)

- **Casos de Auto-Evaluación (CP-01 a CP-03):** Validación de cálculo polimórfico con data semilla (Ana Silva, Carlos Pérez, María Gómez, Luis Rojas), funcionamiento de la Pila LIFO y generación correcta del reporte FIFO en certificados_pendientes.txt.
- **Pruebas Técnicas del Evaluador (EVAL-01 a EVAL-04):** Resistencia a apagones (persistencia física), auditoría de código para confirmar uso de polimorfismo real (sin sentencias if/switch masivas en el main), manejo de excepciones (try-catch) y auditoría de punteros/liberación de memoria en C++.